



Docker Registry GHCR과 GitHub Actions

학습 목표

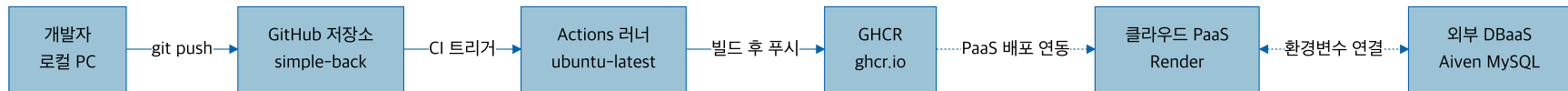
- CPU 아키텍처 차이를 이해하고 클라우드 표준인 `linux/amd64` 이미지를 빌드합니다
- 원격 레지스트리 규격(`ghcr.io/<사용자>/<이미지명>:<태그>`)으로 로컬 이미지에 다중 태그를 매핑합니다
- Git 푸시를 트리거로 GitHub Actions 러너가 이미지를 자동 빌드하고 발행하도록 구성합니다
- 발행된 공개 패키지를 인증 없이 pull 하여 러너 산출물의 아키텍처와 동작을 검증합니다

로컬에서 완성한 이미지는 그 PC를 벗어나지 못합니다. 본 실습은 이미지를 **원격 레지스트리에 보관**하는 과정을 **CI 파이프라인에 위임**하여, 팀과 클라우드가 언제든지 가져다 쓸 수 있는 공용 아티팩트로 만듭니다.

핵심 개념 및 구조

컨테이너 배포 파이프라인 전체 구조

소스 변경을 감지해 이미지를 자동 빌드하고 레지스트리에 안전하게 보관하는 흐름입니다.



🔗 소스 변경

커밋 푸시가 파이프라인의 유일한 시작 신호

🔧 자동 빌드

사람의 개입 없이 러너가 동일한 절차로 이미지 생성

🚀 런타임 실행

운영 서버는 빌드하지 않고 완성된 이미지를 pull 해 실행

컨테이너 레지스트리의 역할

레지스트리는 이미지 레이어를 보관하고 태그로 조회할 수 있게 제공하는 원격 저장소입니다.

구분	로컬 이미지 스토어	원격 레지스트리
위치	내 PC의 도커 엔진 영역	네트워크로 접근하는 원격 저장소
조회 명령	<code>docker images</code>	<code>docker pull</code> 로 다운로드 후 조회
공유 범위	해당 머신 내부로 한정	권한을 가진 모든 머신과 서비스
대표 구현	없음	<code>docker.io</code> , <code>ghcr.io</code> , AWS ECR

현실 개념으로 보면 레지스트리는 **앱스토어**에 해당합니다. 개발자는 완성한 설치파일을 등록하고 (`docker push`), 실행할 쪽은 그것을 내려받아 실행합니다 (`docker pull`). 실행 주체가 소스코드나 빌드 도구를 갖출 필요가 없습니다.

Docker Hub와 GHCR 비교

비교 항목	Docker Hub	GitHub Container Registry (GHCR)
엔드포인트	<code>docker.io</code> (기본값)	<code>ghcr.io</code>
인증 체계	별도 계정·토큰 발급 필요	GitHub 계정 및 PAT(개인용 액세스 토큰) 일원화
CI 자동화	외부 시크릿 수동 등록	내장 <code>secrets.GITHUB_TOKEN</code> 자동 인증
소스 연계	코드와 이미지가 별개 플랫폼에 분리	저장소 <code>Packages</code> 탭에서 일체형 관리
전송 한도	무료 계정 pull 횟수 제한 엄격	GitHub 생태계 내부 통신으로 여유 있는 쿼터

본 차시가 GHCR을 선택하는 이유는 **소스와 이미지를 같은 권한 체계 안에서 관리**할 수 있기 때문입니다. Actions 워크플로가 별도 자격증명 없이 곧바로 레지스트리에 푸시할 수 있습니다.

GHCR 이미지 참조 규칙

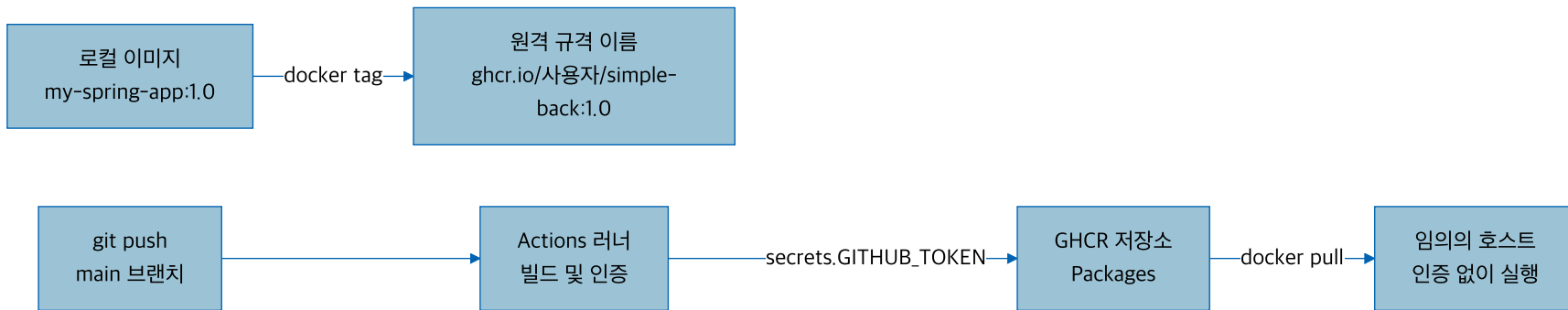
ghcr.io/[사용자]/[이미지명]:[태그]

구성 요소	예시	의미
레지스트리 호스트	ghcr.io	생략 시 docker.io 로 간주되어 GHCR에 도달하지 않음
네임스페이스	본인_계정명	소유자 계정 또는 조직 (반드시 소문자)
이미지명	my-spring-app	저장소 내 패키지를 식별하는 이름
태그	1.0, latest	불변 릴리스 버전 또는 환경 포인터

도커 이미지 이름 규칙상 대문자를 허용하지 않습니다. GitHub 계정명에 대문자가 있으면 푸시 시점에 거부되므로 반드시 소문자로 변환해 사용합니다.

이미지 발행 경로: 태그 매핑과 러너 발행

로컬에서는 원격 규격의 이름을 매핑해 두고, 실제 업로드는 러너가 수행합니다.



태그는 별칭이지 사본이 아닙니다

`docker tag`는 같은 이미지 ID에 이름표를 하나 더 붙일 뿐이라 디스크를 추가로 쓰지 않습니다.

발행 주체는 러너입니다

로컬 자격증명 없이 워크플로 내장 토큰으로 업로드되므로, 로컬에 장기 유효 토큰을 보관하지 않습니다.

태그 전략과 가시성 설정

태그	성격	실무 용도
<code>1.0</code>	불변 릴리스 식별자	배포 추적과 롤백의 기준
<code>prod</code>	환경 별칭	운영 환경이 가리키는 대상을 전환
<code>latest</code>	가변 포인터	편의용 별칭. 운영 배포 기준으로는 부적합

러너가 처음 발행한 패키지는 **비공개** 상태입니다. 외부 서버나 배포 환경에서 별도 레지스트리 자격증명 없이 이미지를 pull 하려면 패키지 설정(`Package settings`)에서 가시성을 `Public`으로 전환해야 합니다. 공개 전환 시 이미지 레이어가 외부에 공개되므로 자격증명이 이미지에 포함되지 않았는지 먼저 확인합니다.

레지스트리 인증 모델 비교

인증 수단	폴네임 / 발급 주체	유효 범위	실습에서의 위치
<code>secrets.GITHUB_TOKEN</code>	워크플로 실행 시 자동 발급되는 임시 토큰	해당 실행 동안만 유효	이미지 발행의 표준 경로
PAT	Personal Access Token (개인용 액세스 토큰)	만료일까지 계정 전반	로컬 수동 <code>docker login</code> 경로, Appendix 참고

🔒 임시 토큰이 안전한 이유

워크플로 종료와 함께 즉시 폐기되고 로그에 마스킹됩니다. 유출되어도 재사용할 수 있는 시간이 없습니다.

🔑 로컬에 토큰을 두지 않습니다

발행을 러너에 위임하면 `write:packages` 권한의 장기 유효 토큰을 로컬에 보관할 필요가 없어져 유출 경로 자체가 사라집니다.

GitHub Actions 워크플로의 구성 요소

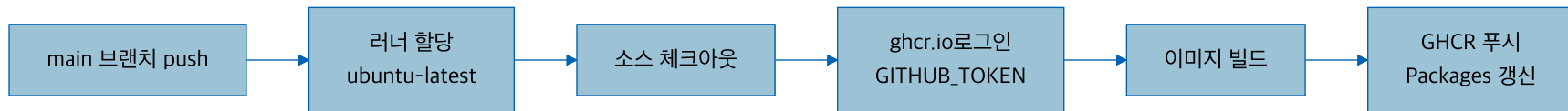
워크플로는 저장소의 `.github/workflows/`에 둔 YAML 파일 하나가 곧 하나의 자동화 정의입니다.

블록	역할	본 차시에서의 값
<code>on</code>	실행 조건 트리거	<code>main</code> 브랜치에 push 될 때
<code>runs-on</code>	작업을 수행할 러너	<code>ubuntu-latest</code> 클라우드 가상 머신
<code>permissions</code>	실행 토큰의 권한 범위	<code>contents: read</code> , <code>packages: write</code>
<code>steps</code>	순차 실행되는 단위 작업	체크아웃, 로그인, 빌드, 푸시
<code>uses</code>	재사용 가능한 공개 액션 호출	<code>docker/build-push-action</code> 등

러너는 매 실행마다 새로 생성되고 종료 후 폐기되는 일회용 머신입니다. 로컬 PC의 설치 상태나 캐시에 영향을 받지 않으므로, 누가 언제 실행해도 같은 절차가 같은 결과를 만듭니다.

워크플로 실행 순서

워크플로는 이벤트가 발생하면 러너를 할당받아 스텝을 위에서 아래로 순차 실행합니다.



🔔 트리거

지정한 브랜치에 변경이 도달하는 순간 자동으로 시작

🖥 러너

GitHub가 제공하는 리눅스 가상 머신에서 단계를 순차 실행

✅ 결과 확인

Actions 탭의 실행 로그와 Packages 탭의 갱신 내역으로 검증

러너는 매 실행마다 초기화된 상태로 할당되므로 로컬에 설치된 도구나 이전 실행의 잔여물에 영향을 받지 않습니다. 빌드에 필요한 것은 모두 워크플로 파일에 선언되어야 합니다.

크로스 플랫폼 빌드 원리

이미지에 담긴 바이너리는 특정 CPU 명령어 집합으로 컴파일되며, 다른 아키텍처에서는 실행되지 않습니다.

실행 환경	CPU 아키텍처	도커 플랫폼 표기
Apple Silicon Mac	ARM64	<code>linux/arm64</code>
Intel 및 AMD PC	x86_64	<code>linux/amd64</code>
클라우드 인스턴스와 리눅스 서버	x86_64	<code>linux/amd64</code>

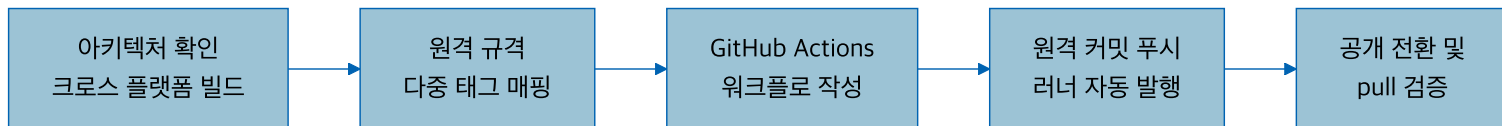
Apple Silicon Mac에서 아무 옵션 없이 빌드한 이미지를 x86_64 클라우드에 올리면 컨테이너 시작 즉시 `exec format error`로 종료됩니다. 해결책은 두 가지입니다. 로컬 빌드 시 `--platform linux/amd64`를 명시하거나, 처음부터 x86_64 러너인 Actions에서 빌드하는 것입니다.

로컬 빌드와 CI 러너 빌드

비교 항목	로컬 PC 빌드	Actions 러너 빌드
아키텍처	개발자 PC에 따라 갈림	항상 x86_64로 고정
재현성	설치된 도구 버전에 영향을 받음	매번 동일한 초기 상태에서 시작
실행 시점	사람이 기억해서 수동 실행	푸시 시점에 자동 실행
소요 자원	개발자 PC의 CPU와 네트워크	GitHub가 제공하는 클라우드 자원

실습에서는 두 경로를 모두 거칩니다. 먼저 로컬에서 `--platform` 을 지정해 아키텍처 문제를 직접 확인하고, 이어서 같은 결과를 러너가 자동으로 만들어 내도록 워크플로로 옮깁니다.

실습 전체 흐름: 태그 매핑부터 파이프라인 검증까지



📁 레지스트리 등록

로컬 이미지에 원격 규격 이름을 매핑해 발행 대상을 확정

🔧 빌드 자동화

워크플로 YAML로 push 시점의 빌드와 푸시를 러너에 위임

🔍 파이프라인 검증

러너 빌드 로그와 패키지 갱신 확인 후 공개 이미지 pull 검증

앞부분에서 이미지 이름 규격과 태그의 성질을 로컬에서 직접 확인한 뒤, 실제 발행은 워크플로 파일로 러너에 위임합니다. 로컬 자격증명이 필요한 절차는 정규 흐름에 두지 않습니다.

실습 단계

실습 로드맵: 이미지 규격 확인과 워크플로 작성

Step	주제	이 단계에서 직접 확인하는 것
1	클론된 저장소 상태 확인	이미지와 함께 형상 관리될 파일의 추적 상태
2	크로스 플랫폼 빌드 점검	호스트 아키텍처와 <code>linux/amd64</code> 명시 빌드
3	GHCR 규격 다중 태그 매핑	재빌드 없이 부여되는 세 개의 원격 태그
4	워크플로 YAML 작성	트리거·권한·빌드 스텝의 선언적 구성

Step 1~2는 로컬 이미지가 클라우드에서 실행 가능한 형태인지 확인하고, Step 3~4는 발행 대상 이름을 확정된 뒤 발행 절차를 선언형 YAML 문서로 정의합니다.

실습 로드맵: 파이프라인 자동화와 검증

Step	주제	이 단계에서 직접 확인하는 것
5	저장소 생성과 커밋·푸시	본인 저장소로의 원격 전환과 트리거 발생
6	Actions 결과와 패키지 공개 전환	러너 자동 빌드·발행 결과와 가시성 설정
7	공개 이미지 pull 검증	인증 없는 pull과 러너 산출물의 아키텍처

클라우드 러너 발행 (Step 5~6)

코드를 푸시하면 러너가 자동으로 구동되어 x86_64 빌드와 GHCR 업로드를 완료합니다.

발행 결과 검증 (Step 7)

공개 전환된 패키지를 인증 없이 pull 하여 매니페스트 플랫폼과 런타임 기동을 확인합니다.

Step 5~7에서는 로컬에서 수행할 수 없는 발행 작업을 [GitHub Actions 러너에 위임](#)하고, 그 산출물을 다시 내려받아 결과를 검증합니다.

명령어 한눈에 보기: 저장소 상태와 아키텍처 점검

클론된 저장소 확인 Step 1

```
# 클론한 스프링부트 프로젝트로 이동
cd ~/workspace/simple-back
ls -la

# 클론 시 설정된 원격 주소 확인
git remote -v

# Dockerfile 및 소스 추적 상태 확인
git status
```

크로스 플랫폼 빌드 Step 2

```
# 호스트 CPU 아키텍처 확인
uname -m

# 멀티 아키텍처 빌드 환경 점검
docker buildx ls

# 클라우드 표준 아키텍처로 명시 빌드
docker build --platform linux/amd64 \
  -t my-spring-app:amd64 .

docker images my-spring-app:amd64
```

Apple Silicon 환경에서 만든 `arm64` 이미지를 `x86_64` 클라우드에서 실행하면 `exec format error`로 컨테이너가 즉시 종료됩니다.

명령어 한눈에 보기: GHCR 규격 다중 태그 매핑

원격 규격 태그 매핑 Step 3

```
export GITHUB_USERNAME=본인_깃허브_아이디

# 고유 버전 태그
docker tag my-spring-app:1.0 \
ghcr.io/$GITHUB_USERNAME/simple-back:1.0

# 운영 환경 태그
docker tag my-spring-app:1.0 \
ghcr.io/$GITHUB_USERNAME/simple-back:prod

# 최신 릴리스 포인터
docker tag my-spring-app:1.0 \
ghcr.io/$GITHUB_USERNAME/simple-back:latest
```

매핑 결과 확인

```
docker images \
ghcr.io/$GITHUB_USERNAME/simple-back
```

확인 항목	기대 결과
IMAGE ID	세 태그가 동일한 값
디스크 사용량	태그 추가로 증가하지 않음

이미지명을 `simple-back`으로 맞추는 이유는 워크플로가 `github.repository` (소유자/저장소명)를 그대로 이미지 경로로 사용하기 때문입니다. 이름이 다르면 로컬 태그와 러너 발행 이미지가 서로 다른 패키지로 분리됩니다. 이 단계에서는 업로드하지 않고 이름 규격만 확정합니다.

워크플로 정의: 트리거와 권한 선언

```
name: Build and Push Docker Image to GHCR
```

```
on:
```

```
  push:
```

```
    branches: [ "main" ]
```

```
env:
```

```
  REGISTRY: ghcr.io
```

```
  IMAGE_NAME: ${ github.repository }
```

```
jobs:
```

```
  build-and-push:
```

```
    runs-on: ubuntu-latest
```

```
    permissions:
```

```
      contents: read
```

```
      packages: write
```

🔗 트리거 조건

`main` 브랜치로 푸시될 때만 파이프라인이 시작됩니다.

🖥️ 실행 환경

러너는 `x86_64` 리눅스이므로 클라우드와 아키텍처가 일치합니다.

🔑 최소 권한

내장 토큰에 소스 읽기와 패키지 쓰기 권한만 부여합니다.

워크플로 정의: 빌드와 푸시 스텝

```
steps:
  - name: Checkout repository
    uses: actions/checkout@v6
  - name: Set up Docker Buildx
    uses: docker/setup-buildx-action@v4
  - name: Log in to GHCR
    uses: docker/login-action@v4
    with:
      registry: ${ env.REGISTRY }
      username: ${ github.actor }
      password: ${ secrets.GITHUB_TOKEN }
  - name: Lowercase image name
    run: echo "IMAGE_NAME=${IMAGE_NAME,,}" >> $GITHUB_ENV
  - name: Build and push
    uses: docker/build-push-action@v7
    with:
      context: .
      push: true
      tags: ${ env.REGISTRY }/${ env.IMAGE_NAME }:latest
```

블록	역할
<code>checkout</code>	러너에 소스와 Dockerfile 배치
<code>setup-buildx</code>	멀티 아키텍처 빌드 엔진 준비
<code>login-action</code>	내장 토큰으로 GHCR 인증
<code>IMAGE_NAME,,</code>	이미지명 소문자 규칙 준수
<code>build-push-action</code>	빌드와 푸시를 한 스텝에서 수행

`secrets.GITHUB_TOKEN` 은 워크플로 실행 중에만 발급되고 종료 시 폐기되므로 저장소에 토큰을 등록할 필요가 없습니다.

명령어 한눈에 보기: 커밋과 원격 저장소 연결

원격 푸시 Step 5

```
git add .
git commit -m "feat: add Dockerfile and GHCR workflow"

# 기본 브랜치명 통일
git branch -M main

# 원격 대상을 본인 저장소로 변경
git remote set-url origin \
  https://github.com/$GITHUB_USERNAME/simple-back.git

git push -u origin main
```

발행 결과 확인 Step 6

확인 위치	기대 동작
Actions 탭	워크플로 실행 결과 녹색 체크
Packages 탭	latest 태그 갱신 시각 변경
Package settings	가시성을 Public 으로 전환

푸시 전 본인 계정에 초기 파일 없는 빈 저장소를 미리 생성해 둡니다. 로컬 레지스트리 인증이 없으므로 [docker logout](#) 절차도 필요하지 않습니다.

명령어 한눈에 보기: 공개 이미지 pull 검증

인증 없는 pull Step 7

```
# 플랫폼 명시 pull (Apple Silicon 필수)
docker pull --platform linux/amd64 \
ghcr.io/$GITHUB_USERNAME/simple-back:latest

# 매니페스트 플랫폼 확인
docker buildx imagetools inspect \
ghcr.io/$GITHUB_USERNAME/simple-back:latest
```

산출물 검증

```
docker images \
ghcr.io/$GITHUB_USERNAME/simple-back

docker run --rm --platform linux/amd64 \
--entrypoint java \
ghcr.io/$GITHUB_USERNAME/simple-back:latest \
-version
```

기본형 워크플로는 `platforms`를 지정하지 않아 러너 아키텍처인 `linux/amd64` 매니페스트만 발행합니다. Apple Silicon 호스트에서 플랫폼을 지정하지 않고 pull 하면 `no matching manifest for linux/arm64/v8`로 실패하므로 `--platform linux/amd64`를 명시합니다.

러너 발행 이미지와 가변 태그의 성질

태그	태그 매핑 직후	공개 이미지 pull 이후
<code>1.0</code> , <code>prod</code>	로컬 빌드 이미지 ID	변화 없음
<code>latest</code>	로컬 빌드 이미지 ID	러너 발행 이미지 ID로 교체

📖 latest는 포인터입니다

같은 이름을 원격에서 내려받으면 로컬 별칭이 원격 이미지로 덮어쓰워집니다. 배포 기준으로 삼기 어려운 이유가 여기서 드러납니다.

📖 러너 산출물은 x86_64입니다

러너가 `linux/amd64` 환경이므로 로컬 아키텍처와 무관하게 클라우드 표준 바이너리가 생성됩니다.

Step별 이해 포인트: 이미지 규격과 워크플로

Step	반드시 이해하고 넘어갈 핵심 포인트
2	이미지는 특정 CPU 아키텍처용 바이너리를 포함한 산출물입니다.
3	태그는 별칭이므로 재빌드 없이 여러 개를 매핑합니다.
3	이미지명이 저장소명과 어긋나면 러너 발행 대상과 다른 패키지가 됩니다.
4	워크플로는 절차가 아니라 트리거·환경·권한을 선언한 문서입니다.

Step 3의 세 태그가 **동일한 이미지 ID**를 가리킨다는 점이 태그가 데이터 복사가 아니라 이름표라는 직접적인 증거입니다.

Step별 이해 포인트: 파이프라인 자동화

Step	반드시 이해하고 넘어갈 핵심 포인트
4	내장 토큰은 실행 중에만 유효하며 저장소에 시크릿을 남기지 않습니다.
5	커밋 푸시 한 번이 로컬 빌드와 레지스트리 업로드를 완전히 대신합니다.
6	패키지가 비공개면 인증 없는 외부 환경에서 이미지를 pull 하지 못합니다.
7	러너가 빌드하므로 호스트 환경과 무관하게 표준 바이너리가 생성됩니다.

러너에서 빌드한 이미지는 항상 `x86_64`이므로 로컬 아키텍처와 무관하게 클라우드에서 실행됩니다. 자동화는 아키텍처 불일치를 예방하는 수단이기도 합니다.

한 장 요약

영역	핵심 요약	비고 / 실무 원칙
레지스트리	소스와 이미지를 GitHub 계정 체계에서 함께 관리	<code>ghcr.io</code> 엔드포인트, <code>Packages</code> 탭 통합
인증 모델	발행은 러너의 내장 임시 토큰으로 일원화	로컬에 장기 유효 자격증명을 두지 않음
빌드 자동화	<code>git push</code> 트리거로 러너가 동일 절차 자동 빌드	플랫폼(amd64) 아키텍처 불일치 예방
운영 배포	레지스트리의 사전 빌드 이미지를 pull 하여 실행	소스 빌드 없이 런타임 환경변수로 설정 주입

컨테이너 배포의 핵심은 **빌드와 실행의 분리**입니다. CI 러너에서 검증된 불변(Immutable) 이미지를 레지스트리에 보관하고, 운영 환경(PaaS/서버)은 소스를 직접 빌드하지 않고 이미지를 pull 하여 런타임 환경변수(DB 접속 정보 등)만 주입받아 즉시 기동합니다.

※ GHCR 이미지를 실제 클라우드 PaaS(Render)에 연결하고 환경변수를 주입하는 구동 절차는 Appendix에서 이어집니다.

Appendix

GitHub Actions 워크플로 계층과 핵심 키워드

키워드	계층	역할 및 문법
<code>name</code>	워크플로	Actions 탭에 표시될 파이프라인 식별 명칭 (<code>name: CI</code>)
<code>on</code>	트리거	실행 이벤트 및 필터 조건 (<code>on: push: branches: [main]</code>)
<code>env</code>	전역/로컬	워크플로 또는 Job/Step에서 공유할 환경변수 선언
<code>jobs</code>	실행 단위	독립 가상 머신에서 구동될 작업들의 집합 (기본 병렬)
<code>runs-on</code>	환경 지정	러너 OS 사양 선택 (<code>runs-on: ubuntu-latest</code>)
<code>permissions</code>	토큰 권한	내장 임시 토큰의 최소 권한 명시 (<code>packages: write</code>)

워크플로는 **Event(트리거) → Job(작업 단위) → Step(실행 단계)** 계층으로 구성됩니다. 각 Job은 독립된 러너에서 병렬로 실행되며, `needs: [job_id]`로 순차 의존성을 구성합니다.

워크플로 Step 실행 문법: uses, with, run

키워드	역할	실무 용도 및 문법 예시
<code>steps</code>	순차 명령	Job 내부에서 위에서 아래로 차례대로 실행되는 태스크 목록
<code>uses</code>	액션 호출	공식/커뮤니티 검증 액션 재사용 (<code>actions/checkout@v6</code>)
<code>with</code>	입력 매핑	<code>uses</code> 로 호출한 액션에 전달할 키-값 인자 매핑
<code>run</code>	셸 명령	러너 가상 머신의 터미널에서 스크립트 직접 실행
<code>id</code>	스텝 식별자	후속 스텝에서 산출물을 참조하기 위한 고유 ID
컨텍스트 표현식	런타임 치환	이중 중괄호 구문으로 <code>secrets.GITHUB_TOKEN</code> , <code>github.actor</code> 등을 주입

자주 쓰이는 공통 작업(코드 체크아웃, 도커 로그인/빌드)은 `uses`로 재사용 액션을 조합하고, 프로젝트 고유 빌드·테스트 스크립트는 `run` 블록으로 직접 실행합니다.

로컬 터미널 수동 push와 PAT

```
# Settings > Developer Settings
# > Personal Access Tokens (Tokens classic)
# 권한: write:packages, read:packages
export CR_PAT=발급받은_토큰_문자열

# 표준 입력으로 전달해 히스토리 노출 차단
echo $CR_PAT | docker login ghcr.io \
  -u $USER --password-stdin

docker tag my-spring-app:1.0 \
  ghcr.io/$USER/simple-back:1.0
docker push ghcr.io/$USER/simple-back:1.0
```

단계	요점
토큰 발급	패키지 권한만 가진 Classic 토큰
보안 로그인	파이프로 전달해 히스토리 미기록
태그·업로드	로컬 이미지를 직접 전송

정규 실습에서 이 경로를 제외한 이유는 두 가지입니다. 로컬 아키텍처 (**arm64**)로 빌드한 이미지를 그대로 올리면 x86_64 서버에서 **exec format error**가 발생하는 문제가 남고, 만료가 긴 PAT가 로컬 환경변수와 쉘 히스토리에 장기간 남을 위험이 있습니다.

클라우드 PaaS 배포와 런타임 환경변수

단계	실습 내용	검증 및 핵심 확인 사항
웹 서비스 생성	GHCR의 사전 빌드 이미지 기반 서비스 생성	소스 컴파일 없이 Existing Image 모드로 즉시 구동
환경변수 주입	외부 DBaaS 접속 정보 및 서비스 포트 설정	이미지 재빌드 없이 런타임에 인프라 설정 결합
부팅 점검	실시간 부팅 로그 및 DB 연결 상태 확인	Pulling image , DB 커넥션 풀 연결 정상 완료
도메인 검증	공인 HTTPS 도메인 요청 및 자원 정리	웹 API 응답 확인 후 무료 티어 서비스 정리

정규 실습에서 구축한 CI 파이프라인(GitHub Actions → GHCR)의 산출물을 클라우드 PaaS(Render)에 연결하여 배포를 완성하는 **심화 자율 실습 과정**입니다.

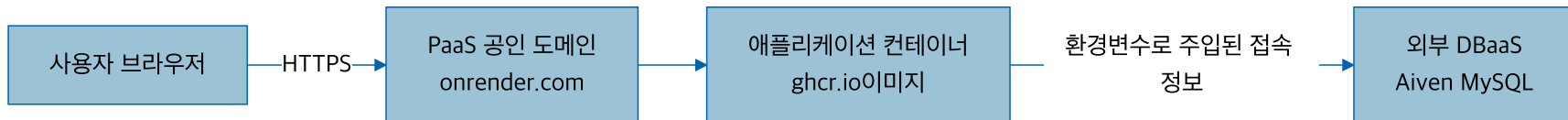
PaaS 배포 모델 비교

배포 방식	빌드 위치	배포 소요	실행 일관성
소스 빌드	PaaS 내부에서 매 배포마다 컴파일	의존성 다운로드까지 포함해 오래 걸림	플랫폼 빌드팩 구현에 의존
사전 빌드 이미지	CI 러너에서 한 번만 빌드	이미지 pull 직후 바로 기동	로컬과 운영이 동일한 이미지

본 실습은 **Existing Image** 모드를 사용합니다. PaaS는 빌드 도구를 갖출 필요 없이 GHCR에서 이미지를 pull 해 실행만 하므로, **CI에서 검증된 산출물이 변형 없이 운영에 도달**합니다.

클라우드 런타임의 상태 분리

컨테이너는 언제든지 종료되고 재생성될 수 있으므로, 보존해야 할 데이터는 컨테이너 바깥에 둡니다.



구성 요소	보관 위치	컨테이너 재생성 시
애플리케이션 이미지	GHCR 레지스트리	동일 태그를 다시 pull
접속 정보와 설정	PaaS 런타임 환경변수	플랫폼이 그대로 재주입
업무 데이터	외부 DBaaS	영향 없이 그대로 유지

세 가지가 모두 컨테이너 바깥에 있으므로 컨테이너는 언제든지 폐기하고 다시 만들 수 있습니다.

배포 환경별 설정 주입 경로

같은 이미지가 로컬과 클라우드에서 모두 동작하려면 접속 정보가 이미지 밖에서 들어와야 합니다.

실행 환경	설정 주입 방법	값이 관리되는 위치
로컬 컨테이너	<code>--env-file</code> 로 파일 지정	형상 관리에서 제외한 환경 파일
Actions 러너	저장소 <code>Secrets</code> 참조	GitHub 암호화 시크릿 저장소
클라우드 PaaS	대시보드의 <code>Environment Variables</code>	플랫폼이 관리하는 서비스 설정

세 경로 모두 값이 이미지 안에 들어가지 않는다는 원칙은 동일합니다. 데이터베이스 비밀번호를 `Dockerfile`이나 소스코드에 적으면, 이미지를 공개로 전환하는 순간 자격증명이 그대로 노출됩니다.

PaaS 웹 서비스 생성과 이미지 연결

설정 항목	선택 값	설명
서비스 유형	Web Service	외부 HTTP 요청을 받는 상시 구동 서비스
배포 모드	Existing Image	소스 컴파일 없이 사전 빌드 이미지를 구동
이미지 주소	ghcr.io/<계정>/my-spring-app:latest	GHCR에 공개된 패키지 경로
인스턴스 사양	Free	실습용 무료 컴퓨팅 자원

소스 저장소를 연결하는 방식은 플랫폼이 매번 빌드를 수행하지만, **Existing Image** 방식은 이미 검증된 이미지를 pull 해 실행만 하므로 배포 시간이 짧고 로컬과 실행 결과가 동일합니다.

런타임 환경변수 주입

```
# PaaS 대시보드 > Environment Variables 등록 항목
PORT=8080
SPRING_DATASOURCE_URL=jdbc:mysql://<Aiven호스트>:<Aiven포트>/defaultdb?sslmode=REQUIRED
SPRING_DATASOURCE_USERNAME=avnadmin
SPRING_DATASOURCE_PASSWORD=Aiven_비밀번호
APP_MESSAGE=Live Cloud Deployment from GHCR
```

☞ 이미지는 그대로

로컬에서 `--env-file` 로 주입하던 값을 플랫폼 설정 화면으로 옮겼을 뿐, 이미지는 재빌드하지 않습니다.

📖 상태는 외부에

컨테이너는 무상태로 두고 데이터는 외부 DBaaS가 보관하므로 재배포해도 데이터가 유지됩니다.

배포 실행과 부팅 로그 점검

로그 키워드	의미	정상 기준
<code>Pulling image</code>	GHCR에서 이미지 레이어 다운로드	인증 오류 없이 다운로드 완료
<code>Started DemoApplication</code>	스프링부트 구동 완료	예외 없이 웹 서버 시작
<code>HikariPool-1 - Added connection</code>	외부 DB 커넥션 풀 연결	커넥션 획득 성공
서비스 상태 <code>Live</code>	헬스체크 통과	공인 도메인 라우팅 시작

`exec format error`가 출력되면 아키텍처 불일치입니다. 러너에서 빌드한 이미지로 교체하거나 `--platform linux/amd64`로 다시 빌드해 푸시합니다.

공인 도메인 검증과 자원 정리

배포 결과 검증

```
export CLOUD_URL=https://본인서비스이름.onrender.com

# 환경변수 메시지 노출 확인
curl -i $CLOUD_URL/

# 외부 DB 조회 결과 확인
curl -i $CLOUD_URL/users
```

실습 자원 정리

```
# 대시보드 > Settings > Suspend Service
# 또는 Delete Web Service 선택

# 로컬 태그 이미지 정리
docker rmi ghcr.io/$GITHUB_USERNAME/simple-back:1.0
docker rmi ghcr.io/$GITHUB_USERNAME/simple-back:prod
docker rmi ghcr.io/$GITHUB_USERNAME/simple-back:latest
```

검증 항목	대상	정상 기준
루트 엔드포인트	<code>\$CLOUD_URL/</code>	<code>APP_MESSAGE</code> 환경변수 값이 응답에 노출
데이터 조회	<code>\$CLOUD_URL/users</code>	외부 DBaaS의 사용자 목록을 JSON으로 반환

무료 인스턴스는 유틸 상태에서 중지되므로 실습을 마친 뒤 서비스를 정리합니다.

레지스트리 선택: 관리형과 자체 호스팅

구분	관리형 SaaS (GHCR)	클라우드 사업자 (ECR)	자체 호스팅 (Harbor)
운영 주체	플랫폼 사업자	클라우드 사업자	조직 내부 인프라 팀
폐쇄망	불가	VPC 엔드포인트로 제한적 지원	내부망 단독 구동 가능
접근 제어	저장소 공개/비공개, 조직 권한	IAM 정책	프로젝트 RBAC, LDAP/OIDC
운영 부담	없음	낮음	높음 (인증서·백업·업그레이드)
주 사용처	오픈소스, 학습, 소규모 팀	해당 클라우드 배포 서비스	규제 환경, 사내 플랫폼

`docker login` · `docker tag` · `docker push` 사용법은 레지스트리 종류와 무관하게 같습니다. ghcr.io/<사용자>/<이미지명>에서 호스트 부분만 harbor.example.com/<프로젝트>/<이미지명>으로 바꿉니다.

Harbor: 저장소를 넘어선 공급망 통제점

Harbor 기능	역할
프로젝트 단위 RBAC	팀·서비스별 논리 분리, 개발자와 운영자 권한 분리
취약점 스캔 연동	Trivy 내장, 심각도 기준 미달 이미지의 pull 차단
이미지 서명 검증	Cosign 서명을 저장·검증해 미서명 이미지 배포 차단
프록시 캐시	외부 레지스트리를 내부에 캐싱, Rate Limit·회선 의존 제거
복제와 보존 정책	리전 간 동기화, 오래된 태그 자동 정리로 스토리지 억제

Harbor는 OCI 참조 구현인 `distribution`을 저장 계층으로 두고 그 위에 인증·스캔·서명·복제를 얹은 CNCF 졸업 프로젝트입니다. `distribution` 단독으로는 인증과 UI가 없어 그대로 운영하기 어렵습니다.

이미지 보안: 내용물과 출처, 두 개의 통제 축

통제 축	막으려는 위협	대표 도구	적용 지점
취약점 스캔	의존성·베이스 이미지의 CVE	Trivy	build 직후
비밀정보 검출	레이어에 남은 자격증명	Trivy secret scan	CI 빌드 단계
SBOM 생성	영향 범위 파악 불가	Syft, CycloneDX	빌드 산출물 첨부
이미지 서명	변조, 태그 덮어쓰기	Cosign	push 시 서명
빌드 출처 증명	빌드 과정의 위·변조	SLSA	CI 산출물 첨부

심각도 기준 초과 시 **종료 코드로 파이프라인을 실패**시켜야 게이트로 기능합니다.

이미지 보안 실무 원칙

⇒ 레이어는 지워지지 않는다

비밀 파일을 `COPY` 한 뒤 다음 레이어에서 `rm` 해도 앞선 레이어에 원본이 남아 복원됩니다. 자격증명은 이미지가 아니라 런타임 환경변수로 주입합니다.

⇒ 공격 표면을 줄인다

경량 베이스 이미지(`alpine`, `distroless`)는 포함 패키지가 적어 CVE 노출 자체가 줄어듭니다. 실행 사용자도 `root` 대신 전용 사용자로 낮춥니다.

✗ 태그는 신뢰의 근거가 아니다

`latest`는 언제든지 다른 이미지로 덮어쓸 수 있는 가변 포인터입니다. 운영 배포는 불변 다이제스트(`@sha256: ...`)로 고정합니다.

✗ 게이트는 push 앞에 둔다

실습에서 구성한 워크플로의 push 스텝 앞에 `aquasecurity/trivy-action`을 두는 것이 최소 구성입니다.

핵심 기술 면접 질문

외부 DBaaS 분리의 필요성

현대 클라우드 아키텍처에서 데이터베이스를 애플리케이션 컨테이너 내부에 두지 않고 외부 DBaaS(Aiven, AWS RDS)로 분리해야 하는 이유는 무엇인가요?

사전 빌드 컨테이너 이미지 배포의 장점

PaaS에서 소스코드를 직접 빌드하는 대신 GHCR의 사전 빌드된 Docker Image를 배포(Deploy Image)할 때의 장점은 무엇인가요?

`exec format error` 원인과 해결책

Apple Silicon Mac에서 빌드한 이미지를 클라우드 서버에 배포했을 때 `exec format error`가 발생하는 원인과 해결책은 무엇인가요?

`secrets.GITHUB_TOKEN` 권장 이유

GitHub Actions에서 외부 PAT(Personal Access Token) 대신 `secrets.GITHUB_TOKEN`을 권장하는 보안상의 이유는 무엇인가요?

핵심 기술 면접 질문

Private Registry 구축이 필요한 경우

SaaS 레지스트리(Docker Hub, GHCR) 대신 Harbor 같은 Private Registry를 사내에 직접 구축해야 하는 상황은 어떤 경우인가요?

취약점 스캔(Trivy)과 이미지 서명(Cosign)의 차이

컨테이너 이미지의 취약점 스캔(Trivy)과 이미지 서명(Cosign)은 각각 어떤 위협을 막기 위한 통제인가요?

모범 답변

외부 DBaaS 분리의 필요성

컨테이너는 언제든지 종료되고 재생성될 수 있는 무상태(Stateless) 아키텍처에 최적화되어 있습니다. 데이터베이스를 컨테이너 내부에 두면 컨테이너 장애나 재배포 시 데이터 유실 위험이 매우 큼니다. 상태(State)를 담당하는 DB는 자동 백업, 고가용성, 복제 기능이 내장된 전용 클라우드 DBaaS(Aiven, RDS)로 격리하고, 애플리케이션 컨테이너는 환경변수로 DB 주소만 참조하도록 설계합니다.

사전 빌드 컨테이너 이미지 배포의 장점

플랫폼 내부에서 매번 소스코드를 컴파일하고 의존성을 다운로드하면 배포 시간이 수 분 이상 지연되고 빌드 실패 가능성이 높아집니다. CI 파이프라인에서 이미 빌드 및 테스트가 완료된 도커 이미지를 레지스트리(GHCR)에 올려두고 PaaS가 단순히 풀(Pull)하여 실행만 하도록 구성하면 배포 속도가 극적으로 빨라지고 로컬과 클라우드의 실행 일관성이 100% 보장됩니다.

`exec format error` 원인과 해결책

CPU 아키텍처(ISA) 불일치로 발생합니다. Apple Silicon Mac은 ARM64 기반이고 대부분의 클라우드 서버는 x86_64 기반입니다. 로컬에서 빌드할 때 `--platform linux/amd64`를 지정하거나, 클라우드 환경(GitHub Actions 러너)에서 이미지를 빌드하도록 파이프라인을 구성하여 해결합니다.

모범 답변

`secrets.GITHUB_TOKEN` 권장 이유

개인용 액세스 토큰(PAT)은 유효 기간이 길고 계정 전반의 권한을 포함할 위험이 있는 반면, `secrets.GITHUB_TOKEN`은 해당 워크플로 실행 중에만 임시 발급되고 종료 시 즉시 폐기됩니다. 또한 YAML에서 `packages: write`와 같이 최소 권한만 선언할 수 있어 자격증명 탈취 위험을 원천 차단합니다. 특히 로컬 개발 환경에 장기 유효한 PAT를 발급받아 보관할 필요 자체가 없어지므로, 토큰이 쉘 히스토리나 로컬 저장소에 남아 유출되는 경로도 함께 제거됩니다.

Private Registry 구축이 필요한 경우

이미지가 영업 비밀이거나 규제 대상(금융·공공·의료)이어서 외부 반출이 금지된 경우, 폐쇄망이라 인터넷 레지스트리에 도달할 수 없는 경우, 대규모 클러스터가 동시 pull 하면서 외부 회선 및 Rate Limit이 병목이 되는 경우입니다. Harbor는 프로젝트 단위 RBAC, 취약점 스캔 게이트, 이미지 서명, 프록시 캐시를 제공해 이미지 공급망 통제점 역할을 합니다.

취약점 스캔(Trivy)과 이미지 서명(Cosign)의 차이

취약점 스캔은 **이미지 안에 무엇이 들어 있는가**를 검사해 알려진 CVE가 포함된 채 배포되는 것을 막는 통제입니다. 반면 이미지 서명은 **이 이미지가 정말 우리 파이프라인이 만든 것인가**를 검증해 레지스트리 계정 탈취나 중간 변조로 악성 이미지가 같은 태그에 덮어씌워지는 공급망 공격을 막는 통제이므로, 두 가지는 대체 관계가 아니라 함께 적용해야 합니다.

깊이 알아보기

컨테이너 레지스트리 Working with the Container registry

GHCR 인증·푸시·가시성 설정 공식 문서

<https://docs.github.com/en/packages/working-with-a-github-packages-registry/working-with-the-container-registry>

워크플로 문법 Workflow syntax

트리거·잡·스텝·권한 키 전체 사양

<https://docs.github.com/en/actions/writing-workflows/workflow-syntax-for-github-actions>

내장 토큰 인증 Automatic token authentication

토큰 발급 범위와 폐기 시점 공식 설명

<https://docs.github.com/en/actions/security-for-github-actions/security-guides/automatic-token-authentication>

빌드·푸시 액션 build-push-action

입력 옵션과 캐시 설정 원문 저장소

<https://github.com/docker/build-push-action>

깊이 알아보기

Private Registry Harbor Documentation

CNCF 엔터프라이즈 레지스트리 설치·운영 문서

<https://goharbor.io/docs/>

레지스트리 규격 Distribution

OCI 레지스트리 참조 구현 원문 문서

<https://distribution.github.io/distribution/>

취약점 스캔 Trivy

컨테이너 이미지·비밀정보 스캐너 공식 문서

<https://trivy.dev/>

이미지 서명 Sigstore Cosign

서명 생성과 검증 절차 공식 개요

<https://docs.sigstore.dev/cosign/signing/overview/>



- EOF -